

AWS Data Pipeline Deployment Guide

This document contains the complete, step-by-step plan for deploying the architecture entirely through the AWS Console, including deploying the starter CloudFormation template.

Phase 1: Deploy the Starter Template

This phase sets up your **Producer Lambda** and the **Ingestion SQS Queue** directly via the AWS UI.

Step 1: Upload to CloudFormation

1. Download or copy the `template_starter.yaml` file from your source repository to your local computer.
2. Navigate to the **CloudFormation Console** in your AWS account.
3. Ensure you are in the **eu-west-1** region (check the region selector in the top right corner of the AWS Console).
4. Click **Create stack** and select **With new resources (standard)**.
5. Under **Prepare template**, ensure the button says **Choose an existing template**.
6. Under **Specify template**, select **Upload a template file**.
7. Click **Choose file**, select the `template_starter.yaml` file you downloaded, and click **Next**.

Step 2: Specify Stack Details & Tags

1. **Stack name:** Enter `mdp-poc-ingestion`. Click **Next**.
2. On the **Configure stack options** page, look for the **Tags** section. Apply your cost tag at the stack level:
 - **Key:** `cost_tag`
 - **Value:** `mdp_poc`
3. Scroll to the bottom and click **Next**.

Step 3: Acknowledge Capabilities (Transforms)

1. Because you are using an AWS SAM template (which uses the `AWS::Serverless` transform), you will see a notice about access capabilities:

Transforms might require access capabilities. A transform might add Identity and Access Management (IAM) resources that could provide entities access to make changes to your AWS account. If a transform adds IAM resources, you must acknowledge their capabilities to create or update them. Ensure that you want to create or update the IAM resources, and that they have the minimum required permissions. In addition, if they have custom names, check that the names are unique within your AWS account.

2. You must check the following boxes to proceed:

- **I acknowledge that AWS CloudFormation might create IAM resources.**
- **I acknowledge that AWS CloudFormation might create IAM resources with custom names.**
- **I acknowledge that AWS CloudFormation might require the following capability: CAPABILITY_AUTO_EXPAND**

Step 4: Review and Deploy

1. Review the final details of your stack configuration.
2. Click **Submit**. Wait until the stack status changes to **CREATE_COMPLETE**.

Phase 2: Create the Storage Layer

Replace `<account-id>` with your actual 12-digit AWS Account ID for all steps below.

1. Navigate to the **S3 Console** and click **Create bucket**.
2. Create the following three buckets. For each bucket, keep **Block all public access** enabled, scroll down to the **Tags** section, click **Add tag**, and add **Key:** `cost_tag` | **Value:** `mdp_poc` :
 - `mdp-poc-code-repo-<account-id>`
 - `mdp-poc-raw-data-<account-id>`
 - `mdp-poc-semantic-data-<account-id>`
3. Open `mdp-poc-code-repo-<account-id>` .
4. Click **Create folder** and name it `glue_scripts` .
5. Upload your `transform.py` file inside the `glue_scripts` folder.

Phase 3: Create the Consumer Lambda

1. Create the Function, Apply Tags, and Set Memory/Timeout

1. Navigate to the **Lambda Console** and click **Create function**.
2. **Name:** `mdp-poc-consumer`.
3. **Runtime:** `Python 3.13`.
4. Scroll down to the bottom of the screen to the **Security & governance** section and toggle **Tags** to the "on" position.
5. Add your cost tag right there:
 - **Key:** `cost_tag`
 - **Value:** `mdp_poc`
6. Leave everything else in its default state and click **Create function**.
7. Once created, navigate to the **Configuration** tab.
8. Select **General configuration** on the left menu, then click **Edit**.
9. Change the **Memory** to `512 MB` and the **Timeout** to `30 sec`, then click **Save**.

2. Configure Environment Variables

1. Go to the **Configuration** tab -> **Environment variables**.
2. Click **Edit** and add a new variable:
 - **Key:** `RAW_BUCKET_NAME`
 - **Value:** `mdp-poc-raw-data-<account-id>`

3. Set IAM Permissions

1. Go to the **Configuration** tab -> **Permissions** and click the Role name link to open IAM.
2. Go to the **Permissions** tab. Click **Add permissions** -> **Attach policies** and add:
`AWSLambdaSQSQueueExecutionRole`.
3. Click **Add permissions** -> **Create inline policy**.
4. Choose **JSON** and paste the following (replace `<account-id>`):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject"],
      "Resource": "arn:aws:s3:::mdp-poc-raw-data-<account-id>/*"
    }
  ]
}
```

5. Name the policy **ConsumerS3Write** and save.

4. Add the SQS Trigger

1. Back on the Lambda page, click **Add trigger**.
2. Select **SQS** and choose your **mdp-poc-ingest-queue**.
3. Set **Batch size** to **10** and click **Add**.

5. Add the Code

1. Go to the **Code** tab. Replace the default code with your Consumer Lambda python code:

```

import json
import boto3
import os
import uuid
from datetime import datetime

s3 = boto3.client('s3')
bucket_name = os.environ['RAW_BUCKET_NAME']

def lambda_handler(event, context):
    records = []
    today = datetime.utcnow().strftime('%Y-%m-%d')

    for record in event['Records']:
        # Athena JSON Lines expects newline delimited objects
        body = record['body']
        records.append(body)

    if records:
        # Join with newline character
        file_content = "\n".join(records)
        file_key = f"data/p_ingest_day={today}/{uuid.uuid4()}.json"

        s3.put_object(
            Bucket=bucket_name,
            Key=file_key,
            Body=file_content
        )
        print(f"Wrote {len(records)} records to s3://{bucket_name}/{file_key}")

    return {"statusCode": 200, "body": f"Processed {len(records)} records"}

```

Phase 4: Set Up the Glue Catalog & Athena

1. Create the Database

1. Navigate to the **AWS Glue Console**.
2. Under **Data Catalog**, select **Databases** -> **Add database**.
3. Name it `mdp_poc_database` and click **Create database**.

2. Create the Athena Table

1. Navigate to the **Athena Console** -> **Query editor**.
2. Run the following DDL query to create the raw table with partition projection (replace `<account-id>`):

```
CREATE EXTERNAL TABLE `mdp_poc_database`.`raw_ingest_data` (  
  `event_id` string COMMENT 'from deserializer',  
  `event_time` string COMMENT 'from deserializer',  
  `charger_id` string COMMENT 'from deserializer',  
  `event_type` string COMMENT 'from deserializer',  
  `payload` string COMMENT 'from deserializer'  
)  
PARTITIONED BY (  
  `p_ingest_day` string  
)  
ROW FORMAT SERDE  
  'org.openx.data.jsonserde.JsonSerDe'  
STORED AS INPUTFORMAT  
  'org.apache.hadoop.mapred.TextInputFormat'  
OUTPUTFORMAT  
  'org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat'  
LOCATION  
  's3://mdp-poc-raw-data-<account-id>/data/'  
TBLPROPERTIES (  
  'classification'='json',  
  'projection.enabled'='true',  
  'projection.p_ingest_day.format'='yyyy-MM-dd',  
  'projection.p_ingest_day.interval'='1',  
  'projection.p_ingest_day.interval.unit'='DAYS',  
  'projection.p_ingest_day.range'='2024-01-01,NOW',  
  'projection.p_ingest_day.type'='date',  
  'storage.location.template'='s3://mdp-poc-raw-data-<account-id>/data/  
p_ingest_day=${p_ingest_day}'  
);
```

Phase 5: Create the Glue ETL Job

1. Create the Glue IAM Role and Apply Tags

1. Go to the **IAM Console** -> **Roles** -> **Create role**.
2. Select **AWS Service** -> **Glue**.

3. Attach the managed policy: `AWSGlueServiceRole`.
4. Provide the name `mdp-poc-glue-job-role`. In the **Add tags** step, add **Key:** `cost_tag` | **Value:** `mdp_poc`. Save the role.
5. Open the role, click **Add permissions** -> **Create inline policy**, use JSON, and paste the following policy (replace `<account-id>` with your actual 12-digit AWS Account ID):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::mdp-poc-raw-data-<account-id>",
        "arn:aws:s3:::mdp-poc-raw-data-<account-id>/*",
        "arn:aws:s3:::mdp-poc-semantic-data-<account-id>",
        "arn:aws:s3:::mdp-poc-semantic-data-<account-id>/*",
        "arn:aws:s3:::mdp-poc-code-repo-<account-id>",
        "arn:aws:s3:::mdp-poc-code-repo-<account-id>/*"
      ]
    }
  ]
}
```

6. Name it `mdp-poc-s3-access-for-glue` and save.

2. Configure the Glue Studio Job

1. Navigate to **Glue Studio** -> **ETL jobs**.
2. Select **Script editor**. In the popup, set **Engine** to `Spark` and choose **Start fresh**. Click **Create script**.
3. Once in the editor, click the **Job details** tab:
 - **Name:** `mdp-poc-transform-job`.
 - **IAM Role:** `mdp-poc-glue-job-role`.
 - **Worker type:** `G.1X`.
 - **Requested number of workers:** `2`.

4. Expand **Advanced properties**:

- **Script filename**: `transform.py` .
- **Script path**: `s3://mdp-poc-code-repo-<account-id>/glue_scripts/` .
- **Job parameters**: Add the following keys:
 - `--datalake-formats` : `iceberg`
 - `--raw_bucket` : `mdp-poc-raw-data-<account-id>`
 - `--semantic_bucket` : `mdp-poc-semantic-data-<account-id>`
 - `--database_name` : `mdp_poc_database`

5. Scroll down to **Tags**, click **Add new tag**, and add **Key**: `cost_tag` | **Value**: `mdp_poc` .

6. Click **Save**.

3. Create the Glue Trigger

1. Navigate to the **AWS Glue Console**. In the left-hand menu, under **Data Integration and ETL**, select **Triggers**.

2. Click **Add trigger**.

3. **Step 1: Set trigger properties**:

- **Name**: `mdp-poc-glue-job-trigger-every-10-minutes` .
- **Trigger type**: Select **Schedule**.
- **Frequency**: Choose **Custom** and enter the cron expression:
`cron(0/10 * ? * * *)` .

4. Click **Next**.

5. **Step 2: Choose jobs or crawlers to activate**:

- Select the job `mdp-poc-transform-job` .

6. **Step 3: Review and create**:

- Review the settings, expand **Tags**, and add **Key**: `cost_tag` | **Value**: `mdp_poc` .
- Click **Finish**.

7. **Step 4: Activate trigger**: Select the newly created trigger in the list, click **Actions**, and select **Activate trigger**.

Phase 6: Notifications & Monitoring

1. Create the SNS Topic and Apply Tags

1. Navigate to the **SNS Console** -> **Topics** -> **Create topic**.
2. **Type**: Standard.
3. **Name**: `mdp-poc-job-notifications`.
4. Expand the **Tags** section, click **Add new tag**, and add **Key**: `cost_tag` | **Value**: `mdp_poc`.
5. Click **Create topic**.
6. Click **Create subscription**. Set **Protocol** to Email, enter your email address as the **Endpoint**, and confirm the subscription via the email sent to your inbox.

2. Create the EventBridge Rule and Apply Tags

1. Navigate to the **Amazon EventBridge Console** -> **Rules** -> **Create rule**.
2. **Name**: `mdp-poc-glue-job-monitor` -> Click Next.
3. Under Event pattern, switch to **Custom pattern (JSON editor)** and paste:

```
{
  "source": ["aws.glue"],
  "detail-type": ["Glue Job State Change"],
  "detail": {
    "jobName": ["mdp-poc-transform-job"],
    "state": ["SUCCEEDED", "FAILED", "TIMEOUT", "ERROR"]
  }
}
```

4. Click Next. For the **Target**, choose **AWS service** -> **SNS topic** -> select `mdp-poc-job-notifications`.
5. Expand **Additional settings** -> **Configure target input** -> choose **Input transformer**.
6. **Input Path**:

```
{"jobState": "${detail.state}"}
```

7. **Input Template**:

```
"Your GlueJob <jobState>!"
```

8. Click Next until you reach the **Review and create** step.
9. Scroll down to the **Tags** section, click **Add new tag**, and add **Key:** `cost_tag` | **Value:** `mdp_poc` .
10. Click **Create Rule**.